Assignment 1

Query Optimization & Data Quality

Covers	Indexing, and what a query plan is telling you · Data quality, deduplication and error routing
Tasks	T1–T7 of the quality workbook, **and the hands-on lab**
Marks	**95** in the workbook + 12 in the paper quick check
CLO	CLO3 · Competence
Component	Hands-on Exercises (20% of the course)
Work	**Individual** — no team roles
Tool needed	**Power BI Desktop, Power Query in Excel, or pandas**
Submit	A **GitHub repository link** on the LMS — folder `assignment1/`
Due	As posted on the LMS

What this assignment is

This is a checkpoint. The midterm is a pipeline and a star, under time, and this is the task that tells you how much of the course you are still carrying. Every claim in T1 comes from a different part of the syllabus, and every one of them is settled by counting something in `data/`.

The ETL quizzes handed you the seventeen steps. This assignment hands you the targets and nothing else.

There is no lab guide, no step list and no M code in this pack. You know how to shape a file, rename to a contract, trim, cast, conform and key — you have done all of it before. How you arrange it this time is your decision, and you defend it in writing rather than matching a walkthrough.

What you are given instead is every number a correct load reaches. They are on the first sheet of the workbook from the moment you open it, on purpose. This is not a guessing game. The question is whether you can build something that arrives at them **and says what it could not do.**

The case

March closed. The same three systems wrote the same three files and the volume went up thirty-five times.

Three things are now true at once. The 08:00 dashboard takes minutes to open. The board has been told PakFreight served **46 customers** in March, and PakFreight has **40**. And some rows in tonight's files cannot be fixed by any transform you know — not by trimming, not by casting, not by conforming. They are simply not loadable, and last month there were none of them.

The earlier rule was that **nothing is dropped silently.** Here that rule becomes arithmetic:

```
rows extracted  =  rows loaded  +  rows rejected  +  rows deduplicated
```

and the audit table that proves it is a deliverable, marked as heavily as the fact table.

The data you need

Everything is in the folder distributed with this assignment.

File	What it is
`data/PakFreight_QualityLab_STUDENT.xlsx`	**Start here.** All seven tasks. Cream cells are yours to fill
`data/march/marine_march.csv`	**261 rows.** IDs as issued, dates as `dd/MM/yyyy` **text**, status UPPER CASE. Structurally clean

File	What it is
`data/march/rail_march.xlsx`	**315 rows.** Two title rows above the header, the `RL-` prefix stripped off every consignment number, weight in **TONNES**
`data/march/road_march.csv`	**708 rows.** IDs lower case, ISO dates, lower-case status with stray spaces, one blank line and one `TOTAL` footer
`data/customers_master.csv`	The customer registry. Read the names before you trust the row count
`data/alias_lookup.csv`	The alias map, **unchanged**: 23 raw strings to 14 cities. March contains strings it has never seen
`data/dims/*.csv`	The star, carried forward. **One of these five files is stale and it is not labelled**
`data/february/*`	February's three files, for the optional refresh test at the end

There is no `DimCustomer`. Building it is part of the assignment.

There is no `FactShipment_March`. You have been handed a fact table before, and you have produced one before. This time you produce one *and* the table that accounts for everything it left out.

Power BI Desktop, Power Query inside Excel (**Data > Get Data**), or **pandas** — all three earn full marks, say which you used. `pip install pandas openpyxl`. Every formula in the workbook is `COUNTIF`, `COUNTA`, `SUM` and `IF`.

The rules

- **Extract corrects nothing.** 1,284 rows arrive and 1,284 rows land, blank line and `TOTAL` footer included.
- **Nothing is dropped silently.** A row that cannot be loaded goes to the reject table **with a reason**. A row whose lookup fails goes to the Unknown member at `-1`, loads, and is **counted**.
- **A reject is not a fix.** Every rejected row must carry enough with it to be re-submitted once somebody corrects the source. "Row 418 failed" is a log, not an audit table.
- **Reject, route and remove are three different decisions.** Getting them confused is most of what this assignment marks.
- **The totals must not move.** Cleaning data never changes what the business shipped.
- **A total is not a row count.** Four of the wrong answers here reconcile to the rupee.
- **Every number in the workbook came out of the files in `data/`.** So can yours.

T1 · The carry-forward audit · 15 marks

Fifteen claims about **this dataset**, drawn from every part of the course so far. For each one: which topic it comes from, true or false, and **the number that settles it**.

A verdict with no number earns nothing. Nothing here is a recall question — every claim can be settled by counting something in `data/`.

Self-check (in the sheet): exactly **3** of the fifteen are true.

> If you have eight or nine trues you are agreeing with statements that sound like good practice instead of checking them against the files. Several of the claims are things you would say yourself, written down carefully enough to be wrong.

Two of the fifteen are the ones to be slowest about: the one about negative numbers, and the one about a business key.

T2 · Profile before you touch · 12 marks

The staging table is **1,284 rows**: shaped and renamed, nothing transformed. Profile all eight contract columns **before** you change a single value.

For each column: blanks, distinct raw values, rows that **fail the stated rule**, and rows that merely **need a text transform** — a correct value in the wrong form.

Self-check (in the sheet): the eight rule-failure counts sum to **105**.

---

105 is not 84 and it is not 24. A row can fail two rules, and a row that fails a rule is not automatically a row you reject. Keeping those two ideas apart is most of the marks on this sheet.

---

6 of the 12 marks are for two written answers: why the profile has to be taken before the transforms rather than after, and which column's distinct count most overstates how bad it is.

Four of these columns look clean and are not. One looks filthy and is fine.

T3 · The reject contract · 18 marks

Nine classes of row and four verdicts: **Load**, **Route to Unknown**, **Reject**, **Remove as duplicate**.

- **Reject** — the row cannot be loaded at all, and goes to `RejectLog` with a reason.
- **Route to Unknown** — the row loads, a lookup failed, the key is `-1`, and the count is published.
- **Remove as duplicate** — the row is not a second shipment, so taking it out restores the grain.
- **Load** — there is nothing wrong with it.

Self-check (in the sheet): **24** rejected, **21** routed to Unknown, **60** removed as duplicates, **3** loaded untouched.

That last number is the one students get wrong. Three rows look exactly like faults and are not — reject them and you have thrown away real business. If your reject count is 27, that is what happened; if it is 38, you rejected the unresolved cities too.

Part B · 8 of the 18 marks

Specify the `RejectLog` schema yourself. List the columns you would give it and say what each one is for. The test is whether somebody who has never seen your pipeline can take a row out of that table, fix the source, and put it through again.

Six columns is a good answer. Four is thin. Nine means you are storing the same thing twice.

Then one written answer: rejecting a row and routing it to Unknown both mean "we could not resolve this", so **why are they not the same decision, and what does each one cost you if you choose the other?**

T4 · The reconciliation identity · 15 marks

Part A · The ladder · 6 marks

Fill it in **from the files**.

Stage	Rows	SUM(WeightKG)	SUM(FreightCostPKR)
`marine_march.csv`, as extracted	?	?	?
`rail_march.xlsx`, as extracted	?	?	?
`road_march.csv`, as extracted	?	?	?
staging, all three appended	?	—	—
rejected	?	—	—
removed as duplicate	?	—	—
loaded into `FactShipment_March`	?	**17,542,050**	**147,037,000**

Two of the weight cells are blank on purpose, and the three per-file figures do not add up to anything. Work out why before you write them in: the rail file is in tonnes, and two of the three files have **text sitting in the weight column**, which `SUM` silently skips.

Then the identity, which must come out at exactly zero:

```
staging − loaded − rejected − removed = 0
```

> Any other number means rows left your pipeline and nothing recorded where they went. That is the only unforgivable outcome here. A wrong number you can find; a missing row you cannot.

Part B · Read the gap · 9 marks

Four students hand in four different pairs of numbers. **Every one of them ran a pipeline that completed without an error.** For each: what did they do, and which check on the T7 sheet catches it?

```
1,212 rows   17,713,950 kg
1,200 rows    8,603,247.75 kg
1,209 rows   17,542,050 kg
1,186 rows   17,346,900 kg
```

> The third one is the sharpest thing on this sheet. The weight is right to the kilogram, the cost is right to the rupee, and nine rows in that fact table are not shipments.

T5 · Duplicates and the distinct count · 12 marks

"Remove duplicates" is not an instruction until somebody says what a duplicate is. Four definitions, applied to the same **1,260** loadable staging rows:

	The definition	Rows surviving
1	the whole row is identical, every column compared	?
2	`ShipmentID` is identical, compared as it arrived	?
3	`ShipmentID` is identical, compared **after** trim and upper case	?
4	same day, same lane, same weight, same customer	?

Three of them are wrong, and they are wrong in two different directions: two keep rows they should drop, one **drops rows it should keep**.

> Definition 4 is the dangerous one, because it sounds more rigorous than the right answer. Find the two pairs of `RD-` consignments it destroys and name them. A business key is a claim about the data; test it before you deduplicate on it.

Part B · 6 of the 12 marks

"How many customers did we serve in March?" Count it four ways: rows in the registry, distinct `CustomerID` in the fact table, distinct businesses after conforming the names, and rows in your `DimCustomer`.

Then: the fact table has **no duplicates** and the customer count is still wrong by six. Where is the duplication, what operation fixes it, and **why do the weight and cost totals not move when you fix it?**

> Note which way each error runs. A duplicate fact row inflates weight and revenue. A duplicate dimension row inflates counts and leaves every additive total exactly right — which is how this one reached the board.

T6 · Index and join path · 15 marks

The fact table is 1,200 rows and the dashboard takes minutes.

Part A · Selectivity · 6 marks

Selectivity = distinct values / rows. It is the fraction of the table a single lookup still has to read. Compute it for all eight candidate columns from your own fact table: `DateKey`, `OriginCityKey`, `DestCityKey`, `RouteKey`, `ModeKey`, `StatusKey`, `CustomerKey`, `ShipmentID`.

Part B · The decision · 5 marks

Which column do you index, and **why that one rather than the most selective one or the most filtered one?** State the cost on write **as a number**.

> `ShipmentID` is perfectly selective — 1.0 — and nobody ever filters a dashboard on it. `ModeKey` is filtered on every page and has three distinct values covering 400 rows each. Neither is the answer, and saying why is the task.

Part C · The join path · 4 marks

One business question — **total weight by province** — down two schemas. Probes are fact rows x joins traversed: the figure a query plan reports as rows passed between operators.

Path	Tables	Joins	Probes
Star: `FactShipment_March` to `DimCity`	?	?	?
Snowflake: `Fact` to `DimCity` to `DimProvince` to `DimCountry`	?	?	?

Star against snowflake has been an argument up to now. Here it is arithmetic.

> A dashboard that takes minutes on 1,200 rows is almost never short of indexes. Before you add one, check the three things this dataset can explain on its own — and an index cannot rescue any of the three.

T7 · Definition of done · 8 marks

Refresh your model, read each number off it, and type it in. **All ten must read "pass".** Submitting with a single `fix` loses all 8 marks.

#	Check	Expected
1	Rows extracted into staging, all three files	**1,284**
2	Rows in `RejectLog`	**24**
3	Rows removed as duplicates	**60**
4	Rows in `FactShipment_March`	**1,200**
5	`SUM(WeightKG)`	**17,542,050 kg**
6	`SUM(FreightCostPKR)`	**PKR 147,037,000**
7	Rows on the Unknown **city** member (–1)	**14**
8	Rows on the Unknown **status** member (–1)	**7**
9	Rows in `DimCustomer`, excluding Unknown	**40**
10	Rows in `DimDate`	**90**

Checks 1 to 4 are the identity. They must satisfy `1 = 2 + 3 + 4` before any of the others mean anything.

Checks 5 and 6 are the famous ones and the weakest on the sheet. Four different wrong pipelines pass check 5 exactly.

Checks 9 and 10 cannot be passed by cleaning the March files at all. They are earlier material arriving to be marked. If check 10 surprises you, open `data/dims/DimDate.csv` and count it.

Checks 7 and 8 are the character question · 4 of the 8 marks

They expect **14** and **7**, not zero. They are the only two checks on the sheet that are satisfied by **admitting something**. Every other check rewards a pipeline that worked; these two reward a pipeline that says what it could not do.

So the sheet asks *how you know* — and asks you to name the unresolved values, and to say what you would have gained by deleting those rows instead.

> Zero was available. Delete the fourteen rows, and the count is 1,186, the dashboard is green, and **195,150 kg** of freight the company actually moved is gone from the quarter. Nobody would have found it.
>
> Routing a row to Unknown is not forgiveness. Deleting one quietly so the model looks complete is a statement about you, not about the data.

The lab · The dual-route load

No walkthrough. The deliverable is two tables and the arithmetic that ties them together:

- **`FactShipment_March`** — 1,200 rows, eight keys and three facts, no descriptive text, the Unknown member at `-1` used where it is needed.
- **`RejectLog`** — 24 rows, the schema you specified in T3, every row carrying its reason and enough of the source to be re-submitted.
- **`DimCustomer`** — 40 businesses and Unknown, with the surviving registry spellings held as aliases, exactly as you did it for cities.
- **`DimDate`** — built for the period, gapless.

If you finish early

Point your pipeline at the three files in `data/february/` and refresh. It should produce 34 rows, 578,650 kg and an **empty** reject table — and if it does, you have built something that works on a month it has never seen, which is the only test of a pipeline that means anything.

Marking guide

Task	What is marked	Marks
T1	Fifteen claims: topic, verdict and **the number that settles it**	**15**
T2	Eight columns profiled, rule failures summing to 105 (6) + why profile before transform, and the overstating column (6)	**12**
T3	Nine classes against four verdicts (10) + the `RejectLog` schema you specify, and reject-versus-route (8)	**18**
T4	The ladder and the identity at zero (6) + four wrong answers read back to their cause and their check (9)	**15**
T5	Four definitions of duplicate, with the pairs definition 4 destroys (6) + the customer count four ways, and why the totals do not move (6)	**12**
T6	Selectivity for eight columns (6) + the index decision, costed as a number (5) + the join path probes (4)	**15**
T7	All ten checks reading **pass** (4) + checks 7 and 8 answered with *how you know*, and the unresolved values named (4)	**8**
	Workbook total	**95**
	Paper quick check, in class	12
Lab	The dual route itself: both tables, the identity, every step named	*gate*

The lab is a gate, not a score. A workbook cannot have passed T7 honestly without the pipeline behind it, and T7's marks go with it.

A verdict with no number earns nothing. That applies to T1 most of all.

How to submit

You submit a link, not a file. Same repository as the quizzes, new folder.

1. Use your course repository:

```
DW-<your roll number>
```

2. Create this folder inside it, exactly:

```
assignment1/
```

3. Put these files in that folder:

File	What it is
`Assignment1_<roll number>.xlsx`	All seven tasks, T7 reading **10 / 10**. A single red cell loses all 8 marks for T7
`pipeline.pbix` — **or** `pipeline.py`	The dual route complete, every step named, the four ladder numbers printed and the ten checks asserted
`FactShipment_March.csv`	**1,200 rows**
`RejectLog.csv`	**24 rows**, every one naming a file, a line and a reason
`DimCustomer.csv`	**40 businesses and Unknown**
`answers.md`	**One page in your own words:** what you rejected and why no transform recovers it; what you routed to Unknown and why that is not the same decision; the `-1` counts as numbers, and the six spellings behind them; the index you would build, and what it costs you on every load
`README.md`	Your name, roll number, and which tool you used

4. Commit and push:

```
git add assignment1
git commit -m "Assignment 1"
git push
```

5. On the LMS, submit the repository link only:

```
https://github.com/<your-username>/DW-<your roll number>
```

Paste it into the text box. **No attachments.**

> **Commit the `.pbix` to your own repository** — it is the deliverable here, so it belongs in the submission.
>
> If your repository is private the submission cannot be marked. Check your own link in a logged-out browser window before you submit it.

The individual quick check is done on paper in class and is not submitted here.

Definition of done

- ☐ The identity holds: **1,284 = 1,200 + 24 + 60**
- ☐ **17,542,050 kg** and **PKR 147,037,000**, exactly
- ☐ All ten checks pass — **10 / 10**
- ☐ Three credit notes in the fact table, as negatives
- ☐ The `-1` counts written down **as numbers**, with the unresolved spellings beside them
- ☐ `RejectLog` has 24 rows and every one names a file, a line and a reason
- ☐ Every transform step **named** — the naming rule has not been repealed
- ☐ Staging queries connection-only; the reject table loads
- ☐ `answers.md` and `README.md` present
- ☐ Pushed, public, and the link opens for a stranger

References

- **Kimball & Ross (2013)**, *The Data Warehouse Toolkit*, 3rd ed. — Ch. 19 on ETL subsystems 4–8 (data cleansing, error event tables, audit dimensions) and Ch. 20 on physical design and indexing.
- **Inmon (2005)**, *Building the Data Warehouse*, 4th ed. — Ch. 3 on the design of the data warehouse and the cost of granularity.
- **Redman (2016)**, *Getting in Front on Data* — on the difference between cleaning data and fixing the process that dirties it. The one-page hand-in is this argument in miniature.